

Projet Python — Groupe 1

Structures de données, récursivité et classes

Enseignante : Rahma GARGOURI — 2026-2027

Présentation finale — Projet en groupe

Vous préparez une **présentation** (PowerPoint, PDF ou autre format) qui répond aux questions ci-dessous, accompagnée d'un **notebook Jupyter** contenant les solutions des exercices, à exécuter pendant la présentation.

Partie A — Structures de données

Questions

1. Quelle est la différence entre une liste, un dictionnaire, un ensemble (**set**) et un tuple ?
2. Expliquez la notion de mutabilité.
3. Quelle est la différence entre une copie superficielle et une copie profonde ?
4. Pourquoi une liste ne peut-elle pas être une clé de dictionnaire, alors qu'un tuple le peut ?

Exercices

A.1 — Fibonacci avec mise en cache. Calculer le n -ième terme de la suite de Fibonacci en stockant les résultats intermédiaires dans une liste ou un dictionnaire, pour ne jamais recalculer deux fois la même valeur.

```
fibonacci(10)    # 55
```

A.2 — Anagrammes. Écrire `sont_anagrammes(a, b)` qui renvoie `True` si `a` et `b` contiennent exactement les mêmes lettres, en ignorant les majuscules et les espaces.

```
sont_anagrammes("chien", "niche")    # True
```

Partie B — Récursivité

Questions

1. Qu'est-ce qu'une fonction récursive ?
2. Quelles sont les deux caractéristiques d'une fonction récursive ?
3. Que se passe-t-il si le cas de base est absent ?

Exercices

B.1 — Aplatir une liste. Écrire `aplatir(liste)` qui transforme une liste imbriquée en liste simple.

```
aplatir([1, [2, [3, 4]], 5])    # [1, 2, 3, 4, 5]
```

B.2 — Puissance rapide. Écrire `puissance(x, n)` qui calcule x^n (n entier positif ou nul) de façon récursive, en faisant beaucoup moins de multiplications qu’une version simple qui multiplie x par lui-même n fois.

<code>puissance(2, 10)</code> <code># 1024</code>

Combien de multiplications fait votre fonction, comparée à la version simple ? Quelle est la complexité de chacune ?

Partie C — Classes

Questions

1. Qu’est-ce qu’une classe ?
2. Quelle est la différence entre une classe et un objet ?
3. Donnez quatre raisons pour lesquelles on utilise de préférence la programmation orientée objet.
4. Pourquoi dit-on qu’en Python « tout est objet » ?
5. Quelle est la différence entre un attribut de classe et un attribut d’instance ?

Exercices

C.1 — Attribut de classe entier ou liste. Écrire une classe avec un attribut de classe de type entier, et une autre classe avec un attribut de classe de type liste. Pour chacune : créer deux objets, modifier l’attribut depuis le premier objet (par exemple avec `+= 1` pour l’entier et `append` pour la liste), puis afficher la valeur vue par le second objet. Expliquer la différence entre les deux cas. Pourquoi faut-il faire attention quand on utilise un attribut de classe d’un type mutable ? Donner un exemple concret.